

好的，這三者的角色和關係如下：

1. Django:

- **角色:** Python Web 框架。負責處理應用程式的核心邏輯，例如：處理業務規則、資料庫互動 (透過 ORM)、URL 路由、使用者認證、HTML 模板渲染等。它是你網站或應用程式的大腦。
- **不擅長:** 直接處理大量並發的網路請求、高效地提供靜態檔案 (CSS, JavaScript, 圖片)。

2. Gunicorn:

- **角色:** WSGI (Web Server Gateway Interface) HTTP 伺服器。它是 Nginx (或其他網頁伺服器) 和 Django 應用程式之間的橋樑。
- **功能:**
 - 接收來自 Nginx 的請求。
 - 管理多個 Django 應用程式的工作程序 (worker processes)，實現並發處理。
 - 將請求轉換成 Django 能理解的 WSGI 格式。
 - 將 Django 的回應傳回給 Nginx。
- **為何需要:** Django 內建的開發伺服器不適合生產環境，Gunicorn 提供了更穩定、高效能的運行方式來執行 Python Web 應用。

3. Nginx:

- **角色:** 高效能的網頁伺服器、反向代理 (Reverse Proxy) 和負載平衡器。它位於最前端，直接面對來自使用者的請求。
- **功能:**
 - **處理靜態檔案:** 非常高效地直接提供 CSS、JavaScript、圖片等靜態資源，減輕後端 Django/Gunicorn 的負擔。
 - **反向代理:** 將動態請求 (需要 Django 處理的請求) 轉發給 Gunicorn。
 - **負載平衡:** 如果你有多個 Gunicorn/Django 實例，Nginx 可以將請求分發到不同的實例。
 - **SSL/TLS 加密:** 處理 HTTPS 連線。
 - **請求緩衝:** 緩衝慢速客戶端的請求，保護後端應用。

彼此關係 (請求流程):

1. 使用者的瀏覽器發送請求到你的伺服器 (通常是 80 或 443 埠)。
2. **Nginx** 接收請求。
3. **Nginx** 判斷請求類型：
 - 如果是請求**靜態檔案** (如 `/static/style.css`)，Nginx 直接從指定的靜態檔案目錄 (在你的 `docker-compose.yml` 中是 `static_volume`) 讀取並回傳給使用者。
 - 如果是請求**動態內容** (如 `/products/` 或 `/admin/`)，Nginx 將請求**轉發 (proxy pass)** 給 **Gunicorn** (在你的設定中，通常是轉發到 `web` 服務的 8000 埠)。
4. **Gunicorn** 接收來自 Nginx 的請求。
5. **Gunicorn** 將請求透過 WSGI 介面傳遞給 **Django** 應用程式的一個工作程序。
6. **Django** 處理請求：執行對應的視圖函數、查詢資料庫、渲染模板等。
7. **Django** 將產生的 HTTP 回應傳回給 **Gunicorn**。
8. **Gunicorn** 將回應傳回給 **Nginx**。
9. **Nginx** 最後將回應傳回給使用者的瀏覽器。

簡單來說：**Nginx** 是門衛和靜態檔案服務生，**Gunicorn** 是餐廳領班負責調度廚師，**Django** 是廚師負責烹飪 (處理核心邏輯)。